WARWICK

SCHOOL OF ENGINEERING

Lab 1 – Zynq Matrix Multiplication

Module Code:

ES4F3/ES3F1

Module Title:

High Performance Embedded Systems Design

Last Update: Jan 2025

Lab 1

Lab 1 is an introduction to Xilinx's Zynq SoC Architecture using the Digilent Zybo Z7-20 series FPGA boards. We'll walk through setting up the processing system in hardware, exporting this from Vivado to the Vitis IDE and writing C code to run on one of the ARM cores. AMD provides good documentation and a support forum for Vivado and Vitis that can be used if you are encountering an issue. Some relevant documentation is linked at the bottom of this document, but more can be found at docs.amd.com.

Viviado Setup

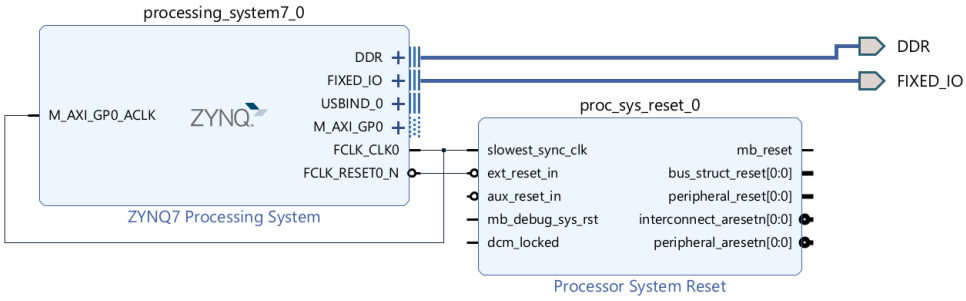
Create Project

- Open Vivado 2024.1
- Under "Quick Start" click "Create Project"
- Set project name and location.
  [Do not use spaces in any Vivado project paths]
- Select "RTL project" and check "Do not specify sources at this time"
- Open the "boards" list and click the "Refresh"
- Search for "Zybo" and click the download icon next to "Zybo Z7-20"
- Select the "Zybo Z7-20" row and finish the setup

Create Block Diagram

- Under "IP INTEGRATOR" click "Create Block Diagram"
- In the diagram add the following blocks (Ctrl+I)
  - ZYNQ7 Processing System
  - Processor System Reset
- Connect `slowest_sync_clk` to `M_AXI_GP0_ACLK` and `FCLK_CLK0`
- Connect `ext_reset_in` to `FCLK_RESET0_N`
- Click "Run Block Automation" in the green banner then click "Ok" with the default values.
- Click the "Regenerate Layout" arrow in the diagram controls bar

- It should look like the diagram below



- Validate the design (F6)
  [You can ignore errors about `PCW_UIPARAM_DDR_DQS_TO_CLK_DELAY` having negative values]
- Save the design (Ctrl+S)
- Under the “Sources” tab find the block diagram in the “Design sources” folder
- Right click the block diagram entry and select “Create HDL Wrapper…”
- Use the default options.

Generate Bitstream

- Run Synthesis
- Run Implementation
- Generate Bitstream
- Under File->Export click “Export Hardware”
- Select “Include bitstream” and finish

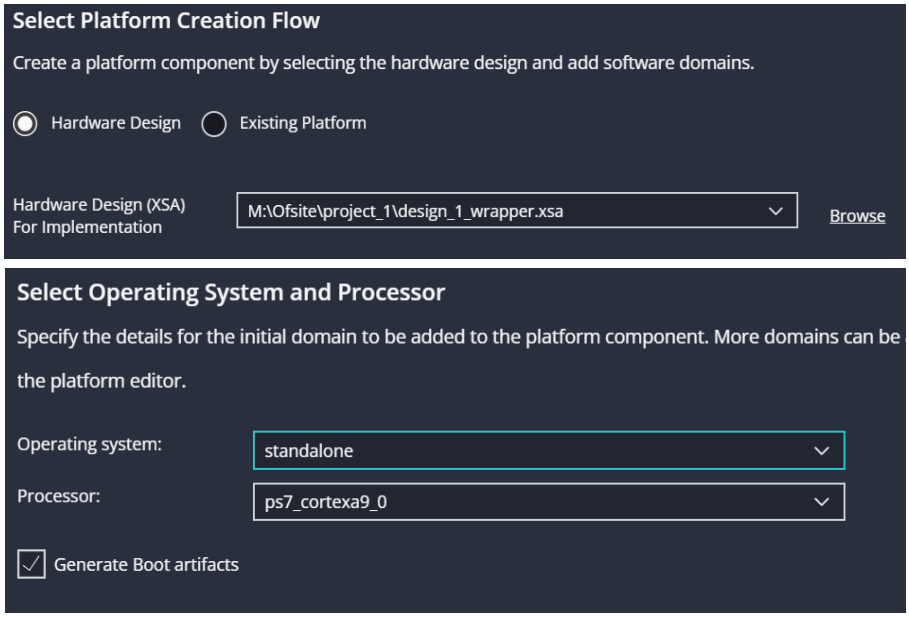
Vitis Setup

- Open Vitis IDE. This can be done through Tools->Launch Vitis IDE
- Click “Open Workspace” and select a new folder to use as the workspace. This could be your Vivado project directory. When you create a Vivado project it creates a new folder within the directory you pick. This is different in Vitis, the workspace folder directly holds all data related to the current project.

A Vitis project is composed of a collection of components. In this lab we will use a platform and an application component. The platform component contains data related to the physical hardware used such as the bitstream and hard processors. The application component is a C application that can be run on the processor defined in the platform component. You can have multiple of the same component types in a workspace, but it is recommended you create a new workspace folder for each lab.

Create Platform

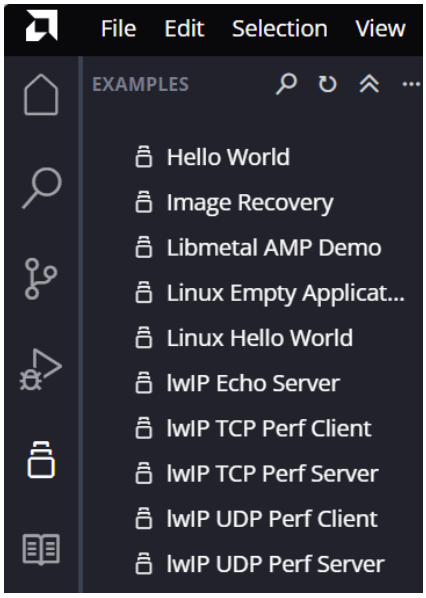
- Create a new platform with File->New Component ->Platform
- Specify a name
- Select hardware design

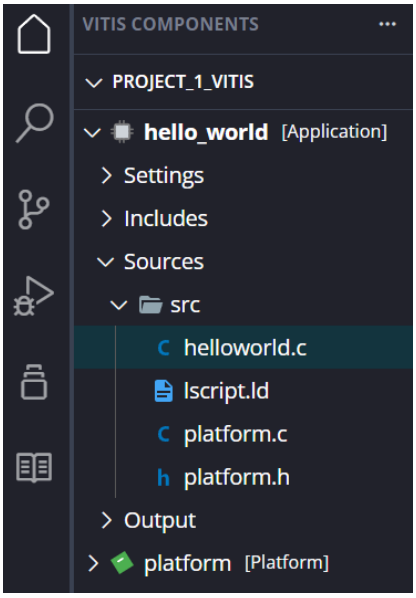


- Click “Browse” and select the XSA file that was created when you exported the hardware from Vivado. This file should be shown in the Vivado TCL console after you have exported it. It should be in the root folder of your Vivado workspace.
- Set Operating system to “standalone”, Processor to “ps7_cortexa9_0” and check “Generate Boot artifacts”
- Click Finish and wait for the platform to be created. The platform config window should appear when it has finished.

Create Application

- Open the list of examples by clicking the icon in the left bar

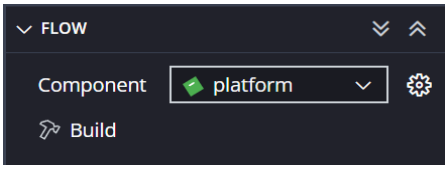


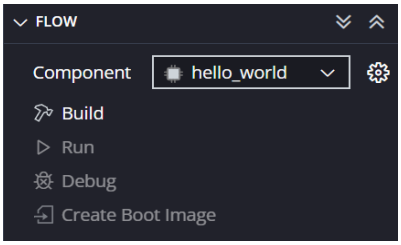


- Select "Hello World" and click "Create Application Component from Template"
- Select the platform you created and finish the application creation process.
- You can view the source code for this example in the "Sources" folder under the applications folder.
  [The "'xil_printf.h' file not found" error can be ignored]

Build and Execution

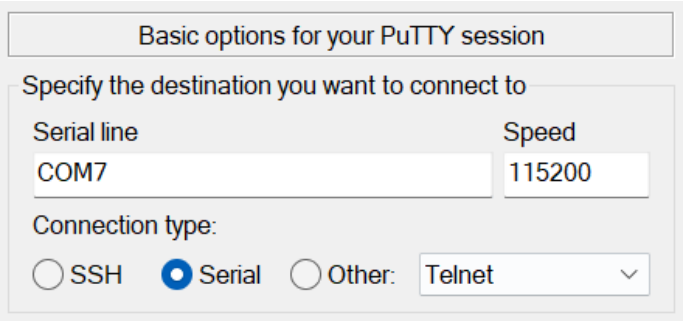
- On the left-hand side, you should see a list of components and a section named "Flow". This Flow section is used for building and running components. Use the dropdown to select your platform component or double click it in the components list, then click "Build". You should see "Platform Build Finished successfully" in the console output.
- Then select the application component and click "Build". "Build Finished successfully" should be printed to the output console.





- Ensure the Zybo device is connect to your PC, is switched on and has the correct jumpers set for USB power and JTAG programming.

- Connect to the serial interface with the PuTTY application.
    - Connection type: Serial
    - Serial line: COM<N>
      [Check in Device Manager to see what COM port your board is using]
    - Baudrate: 115200



- In Vitis click "Run". You should see text printed to the serial output of the device. **[If you click "Run" again it will execute but a warning will appear about an existing debug session. This can be ignored.]**

Tasks

Your task is to modify "hello_world.c" in the sources folder of the application component you created. You will create a program that multiples two matrices and measure the time taken to complete the computation.

Xilinx provides the `xiltimer.h` library that can be used to get the current time as shown below [1]. The difference can then be found to see how long the process took. The value returned will decrease over time so the final value should be deducted from the starting value.

```
usleep(1); // Needed to get timer to work

XTime tStart, tEnd;

XTime_GetTime(&tStart);
// Perform long operation
XTime_GetTime(&tEnd);

printf("\n\rTime start: %llu\n\r", tStart);
printf("Time end: %llu\n\r", tEnd);
printf("Time taken: %lld\n\r", tStart-tEnd);
```

There is a bug that means the processors timer will only start counting after `usleep()` is run. Call this function before `XTime_GetTime()` or it will only output 0.

The print function `xil_printf()` is available in the ` xil_printf.h` library as an alternative to `printf()`. It is much lighter but does not support floating point or long numbers [2]. ` printf()` can be used instead.

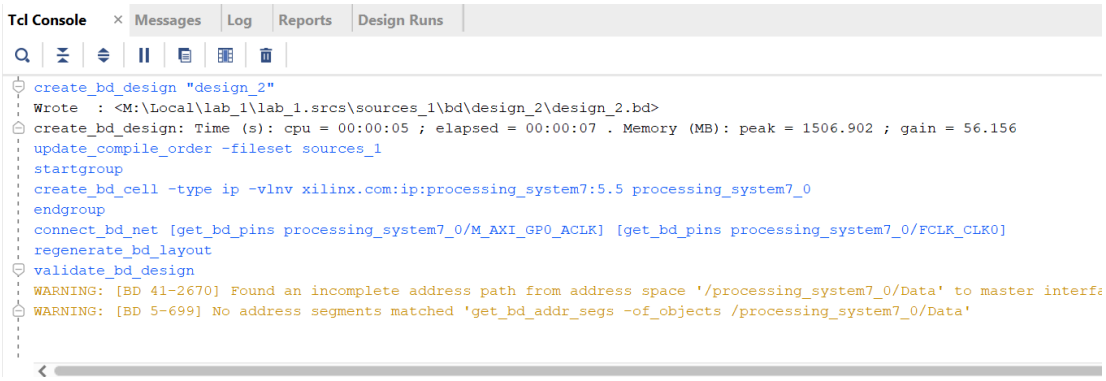
The variable type `XTime` is a U64 int, so `%llu` should be used to print an `XTime` value.

1. Create a C function that takes an integer pointer to a pointer and initialises each of the elements in the 2D array (which represents a matrix) with a random integer using the rand() function. Make sure to allocate memory for each 2D array (both rows and columns!). Don't forget to seed your rand() function with srand(YOUR_SEED_VALUE) to ensure that you don't get the same results each time the program runs.
   Hint: Ideally, you want YOUR_SEED_VALUE to be a different number each time you start your application.
2. Format and print your random matrices to the serial console so you can check the input values against your output matrix later.
3. Allow for a macro to set the size of the input 2D array (i.e. #define SIZE). We'll use this to see the effect on matrix size against the required clock cycles for computation.
4. Write a function that takes 3 array pointers and multiplies the first two together, storing the result in the 3rd.
5. Sample the time before and after the multiplication and calculate the time taken for the computation.
6. Format and print your output solution matrix to the serial console. Do this outside of your timer so as not to affect the timing operation.
7. Don't forget to deallocate memory and run cleanup_platform() at the end of your program to release any used instances from memory

Extension

1. Modify your function to handle 2D arrays that contain floats, how does this affect the computation time?
2. Think about how we might do this multiplication on an FPGA? Could we offload the multiplication from the processing system to the FPGA to improve performance? What are the constraints with working with floating point numbers on an FPGA?

3. In the backend Vivado is a command line application that is controlled through TCL scripts [3]. Every interaction you make with the GUI just runs TCL commands which can be seen in the TCL console. The blue text is executed commands. These commands are very well documented by Xilinx [4].



   Use the TCL Console output and the provided cheat sheet to create a new TCL script that can fully generate the Vivado project for this lab and export the hardware.

   Scripts can be executed through the TCL console or the “Tools” menu.

4. Just like Vivado, Vitis can also be used through scrips and provides a Python API to do this. The Vitis Console application can be used to learn about functions available and is an interactive python shell. The “vitis” module can be imported and then “help(vitis)” to learn more about it. This API is also well documented [5].

   Use this documentation and the cheat sheet in the lab to create a script that creates the Vitis workspace, platform and application component. It should then build all the components.

   To run this script use “vitis -s VitisScript.py” in the terminal. or “run VitisScript.py” in the Vitis Python Console. Fully close the terminal after running this to release the Vitis workspace.
5. Another tool Xilinx provides is the software debugger (XSDB) [6][7]. This can allow you to control the processor through a command line and can be a useful tool for development. There is also a python API for this tool [8].

   Use the documentation and the provided sheet to extend your previous Vitis script to upload the bitstream to the Zybo board and run the application.

References

[1] - https://docs.amd.com/r/2024.1-English/oslib_rm/XilTimer-APIs

[2] - https://docs.amd.com/r/2024.1-English/oslib_rm/xil_printf

[3] - https://wiki.tcl-lang.org/page/Tcl+Tutorial+Lesson+0

[4] - https://docs.amd.com/r/2024.1-English/ug835-vivado-tcl-commands

[5] - https://docs.amd.com/r/en-US/ug1400-vitis-embedded/Vitis-Python-API

[6] - https://docs.amd.com/r/en-US/ug1725-xsdb-reference-guide/Software-Debugger-XSDB

[7] - https://wiki.analog.com/resources/fpga/playing_with_xsdb

[8] - https://docs.amd.com/r/en-US/ug1400-vitis-embedded/Python-XSDB-Commands

Additional resources:

- https://github.com/Xilinx/Vitis-Tutorials
- https://docs.amd.com/r/2024.1-English/Vitis-Tutorials-Embedded-Software/Vitis-Tutorials-Embedded-Software-XD260
- https://docs.amd.com/r/2024.1-English/ug1400-vitis-embedded/Getting-Started-with-Vitis